

GTIN Scanner

A handheld scanning app that replaced a legacy Excel macro for supply-chain receiving — built for the pace and unreliability of a real warehouse floor, not a desk.

Prepared for Directors, stakeholders & IT **Status** Live pilot, on free hosting

Data source Microsoft Fabric Lakehouse (live since Aug 7, 2026) **Updated** Aug 31, 2026

01 · OVERVIEW

What it replaces, and why it exists

Pickers used to scan barcodes into an Excel macro to check items against contract lines. It worked, but it lived on one machine, had no audit trail, and gave supervisors no visibility into what was happening on the floor in real time.

The GTIN Scanner is a web app pickers open on a handheld device. They scan a badge to start a session, then scan items — the app looks each one up instantly against the current supply contract and flags anything on hold. Supervisors watch every active session from a separate monitor board and can export or close any of them.

184,411

contract lines indexed for
instant lookup

O(1)

lookup time per scan —
dictionary, not a search

3 days

session data retention window
(export before then)

From an Excel macro to a modernization project

This didn't start as a request for a new app — it started as one team's Excel-macro barcode tool that had quietly become load-bearing for receiving. It only runs on one machine, keeps no history once the sheet is closed, and gives nobody outside that machine any visibility into what's been scanned today. It was picked up as a small modernization effort: rebuild the same workflow as a real web app, on the company's actual data platform, without changing what a picker's day looks like.

PHASE 1

Prove the core lookup works

Barcode in, contract-line match out, with a public FDA device database as a fallback for anything not on the current contract. Built and tested against a mock dataset first, so the workflow could be validated before touching any live company system.

PHASE 2

Connect to real company data

Wired up to the organization's actual data platform in place of the originally planned one (see "Dead ends & pivots" below) — the contract-line catalog now loads from the live Microsoft Fabric Lakehouse.

PHASE 3

Make it a real multi-user tool

Split into a handheld surface for pickers and a monitor board for supervisors, both reading and writing the same session/scan history, so a supervisor can see today's activity without walking the floor.

NOW

Tuned against real hardware, seeking a permanent home

Scanning behavior has been corrected to match the actual handheld scanners used on the floor. See "Where it runs today" below for the current hosting setup.

Two screens, one shared record

Every session and scan writes to the same Postgres database, so the handheld and the monitor board are always looking at the same truth — there's no sync delay and nothing lives only in one browser tab.

PICKER

Handheld

Scans a badge and a location to start a session, then scans items in a loop. Only the device that started a session can end it normally.

SUPERVISOR

Monitor board

A live list of today's sessions — who, where, how many scans, last activity — with drill-in, per-session export, and a Force End for a device that dropped off.

MANAGER

Admin page

Email-allowlisted refresh control for pulling the latest contract data on demand, plus the same Force End control.

Local first, internet second

This is the core reliability decision in the app: the contract catalog is loaded into memory once, so **the large majority of scans never touch the network at all**. The internet is only involved for the rare item that isn't on the current contract.

1 · SCAN Barcode read, GS1 wrapper parsed, GTIN extracted.	2 · LOCAL LOOKUP Checked against the in-memory contract index. No network call.	3A · HIT Item, on-hold status, and unit-of-measure shown immediately.	3B · MISS Falls back to the FDA's public device-identifier API.
--	---	---	---

If that fallback call also fails — a timeout, a bad connection — the app records it as a clean "Not Found" instead of erroring out, and logs *why* it missed (wrong check digit, not on the current contract, inactive line, etc.) so the miss is explainable later, not just a blank.

EXAMPLE SCAN	ITEM	RESULT
00841098765432	Surgical Mesh 15cm	Found — contract line
00123456789012	Nitrile Gloves, Large	Found — on hold warning
00000000000000	Not on contract	FDA API fallback

Built for the floor, not the demo

Every feature below exists because of something that broke, stalled, or annoyed someone during real use — not a hypothetical.

Hands-free auto-submit

SPEED

PROBLEM

The warehouse's actual scanners (Zebra handhelds) deliver a whole barcode in one instant burst, not as typed keystrokes. The first version of auto-submit was tuned for a keystroke-by-keystroke scanner and simply never fired on the real hardware — pickers had to tap the screen after every item.

SOLUTION

Two independent triggers now cover both delivery styles: scan-speed cadence for guns that type character-by-character, and a check-digit validity test as a backstop for guns that paste the whole code at once. Manual typing still works — Enter or the Look Up button always did.

Audible confirmation

SPEED

PROBLEM

Pickers are watching the item in their hands, not the screen. A quiet or ambiguous tone was easy to miss over warehouse noise, or read as a glitch rather than a confirmation.

SOLUTION

A distinct two-note chime on every successful scan, with separate alarm tones for on-hold and not-found results — loud and clear enough to register without looking up, and unmistakably different from an error.

Session resume after a dropped connection

RELIABILITY

PROBLEM

Warehouse wifi is inconsistent, and mobile browsers routinely kill or reload a backgrounded tab. Losing the tab used to mean losing the session and starting over.

SOLUTION

Every session and scan is written straight to the database, not just held in the browser tab. The session ID lives in the page URL, so a refresh, a killed tab, or a full browser restart reopens the same session with its full scan history intact — nothing is lost, and no scan has to be redone.

Graceful network fallback

RELIABILITY

PROBLEM

The fallback lookup depends on an external, public FDA database over the internet — a connection that isn't always available or fast on a warehouse floor.

SOLUTION

A network failure there never crashes the scan — it resolves to a clean "Not Found" with the reason logged, so the picker keeps moving and the miss is still traceable afterward. And because most scans hit the local index first, this fallback is rarely exercised at all.

Fast cold start

SPEED

PROBLEM

Loading the full 184k-row contract index used to happen before the app showed anything, adding a multi-second blank wait every time the app woke up.

SOLUTION

The index now loads lazily, in the background, after the picker's start screen is already on screen — visible progress instead of a blank wait, with no change to how fast lookups run once scanning starts.

On-hold and duplicate-scan awareness

DATA QUALITY

PROBLEM

A found item isn't automatically a safe item, and rescanning the same barcode twice used to either get silently dropped or inflate the count as if it were a new item.

SOLUTION

Items flagged on hold in the contract data show a distinct warning instead of a plain match. A repeat scan of something already in this session is recognized instantly (no re-lookup) and its count increments, so exports reflect exactly what was physically scanned.

Explainable misses

DATA QUALITY

PROBLEM

A blank "Not Found" gives a supervisor nothing to act on — is the item genuinely new, mislabeled, or just off-contract?

SOLUTION

Every miss is diagnosed and tagged with a specific reason (bad check digit, not on the current contract, inactive line, etc.), stored with the scan and surfaced in both the app and the Excel export.

What we tried first, and why it changed

Three decisions in this app were not the first thing tried. Each is here because the first approach ran into a real constraint — worth knowing before anyone assumes the current design was the obvious starting point.

Contract data source: Redshift → Fabric Lakehouse

Blocked by access, not by engineering

TRIED	Amazon Redshift, the data platform originally assumed to hold the contract-line data.
WHY IT STALLED	Redshift credentials were requested but never issued — the org's actual live data platform is a Microsoft Fabric Lakehouse, not Redshift.
WHAT SHIPPED	The loader was rebuilt against Fabric's SQL analytics endpoint (ODBC, ~184k contract lines), authenticated with Azure AD instead of a stored password. Live since Aug 7, 2026.

Session storage: JSON file → SQLite → hosted Postgres

Two rounds — the first fix wasn't enough either

TRIED	A single JSON file autosaved to local disk, then a local SQLite database, as the record of every session and scan.
WHY IT STALLED	Streamlit Community Cloud restarts the app's container on its own schedule, and its local disk is wiped on every restart — both approaches silently lost every session and scan on the next restart, mid-shift or not.
WHAT SHIPPED	A hosted Postgres database (Neon) that lives outside the app's container entirely, so an app restart is invisible to a picker's in-progress session. A daily scheduled job handles data retention independently of whether the app is even running.

Badge/session linking: in-app camera QR scan

Scoped out before it was built

- | | |
|-----------------------|---|
| TRIED | An early design considered scanning a QR code with the device camera, inside the app, to link a handheld to a session. |
| WHY IT STALLED | Streamlit's UI runs custom components inside an iframe, and iframe-embedded components can't reliably get camera (<code>getUserMedia</code>) permission across devices and browsers — this would have spent the riskiest engineering effort in the project on a problem that turned out to have a simpler answer. |
| WHAT SHIPPED | Sessions are minted directly on the handheld from a typed/scanned badge ID and location — no camera, no QR, no in-app scanning step at all. |

Fabric sign-in: interactive Azure AD → service principal

Works for development today; still open for a hosted deployment

- | | |
|----------------------------------|--|
| TRIED | Interactive Azure AD sign-in (device-code or browser popup) — a human approves the login once, then a cached token is reused for about a month. |
| WHY IT STALLS FOR HOSTING | Fine for a developer's laptop, but a headless host has no human available to approve a browser prompt — the device-code and browser flows simply cannot complete unattended. |
| WHAT'S NEEDED | A service-principal credential (tenant ID, client ID, client secret) that signs in with no human step — supported in the app already, but the credential itself has to be issued and hosted somewhere safe. This is the central item in the IT conversation below. |

What's stored, and for how long

Sessions and scans are kept for **3 days**, then purged automatically by a daily scheduled job — this runs independently of the app, so it fires even if nobody has the app open. Export any session you need to keep before then; there's no longer-term archive in this version.

Each session can be exported to a single Excel workbook from the monitor board — one file per session, named with the location, picker, and date so a batch of exports stays sortable.

Who can do what

Anyone can pick up a device and start a scanning session — that's the front-line workflow and it's intentionally open. Two actions are restricted to a named allowlist of managers and supervisors: refreshing the contract data on demand, and force-ending a session that a device dropped without properly closing.

Note on the access model: admin access today is a typed-email allowlist check, not a verified login — closer to a sign-in sheet than a badge reader. Every attempt, granted or denied, is logged, so misuse is visible after the fact even though it isn't blocked up front. This is a known, deliberate trade-off for now; see Open Items.

Current hosting and technical footprint

The app and its data connections are real and live. Today it runs on **Streamlit Community Cloud**, a free public hosting tier meant for demos and personal projects: the app's source deploys straight from GitHub, sleeps after inactivity, and runs on shared infrastructure outside the organization's own systems. It has been sufficient to build and pilot the tool.

Every scan and session a picker records — who scanned what, when, and from where — is written straight to a **Neon Postgres database**, a hosted cloud database separate from both Streamlit and the Fabric catalog. That's what makes a session survive a dropped connection or a restarted app (see "Session resume after a dropped connection" above): the record lives outside the app itself, not in the picker's browser.

APPLICATION

Framework
Python 3.11+ / Streamlit

Current host
Streamlit Community Cloud

Source control
GitHub (private repo)

Deploy model
Push to deploy

DATA CONNECTIONS

Product catalog (contract lines)
Microsoft Fabric Lakehouse, via ODBC

Catalog sign-in
Azure AD (interactive today)

Scan & session records
Neon Postgres (hosted database)

Fallback lookup
Public FDA device database (no login needed)

BACKGROUND JOBS

Old data cleanup
Daily, GitHub Actions

Catalog refresh
Manual button + 24h auto-refresh

SECRETS

Local dev
.env / secrets.toml (gitignored)

Hosted today
Streamlit Cloud secrets store

Needed for hosting
Org secret manager / vault

Possible future directions

- 1 **Internal hosting.** The app could move to an internal VM/container platform or Azure App Service, since it's already inside the Azure AD tenant for the Fabric connection, with outbound network access to the Fabric SQL endpoint and to Neon (or an internal Postgres instance).
- 2 **Unattended sign-in to Fabric.** A service-principal app registration in Azure AD, scoped read-only to the contract-line table, would let a hosted app authenticate on its own instead of a person approving a login prompt.
- 3 **Managed secrets.** The service-principal secret, the database connection string, and the admin allowlist could move into a proper secret store instead of the app's own hosting config.
- 4 **Session/scan storage location.** Neon is already working well; an internal Postgres instance is an option if data residency ever requires storage to stay inside the network.
- 5 **Verified admin access.** Today's typed-email allowlist could be replaced by real sign-in against the company's Microsoft identity; see "Access & audit" above.

10 · OPEN ITEMS

What's next

- **Internal hosting.** Moving off Streamlit Community Cloud onto internal infrastructure. See "Where it runs today" above for the current footprint.
 - **Verified admin login.** Replace the typed-email allowlist with real sign-in through the company's existing Microsoft identity — same provider already used for the contract-data connection, so no new system to stand up.
 - **Web app team hand-off.** Looping in the broader web app development team as the next step for longer-term ownership.
-

GTIN Scanner — internal supply-chain tooling. This brief reflects the app as of Aug 31, 2026.